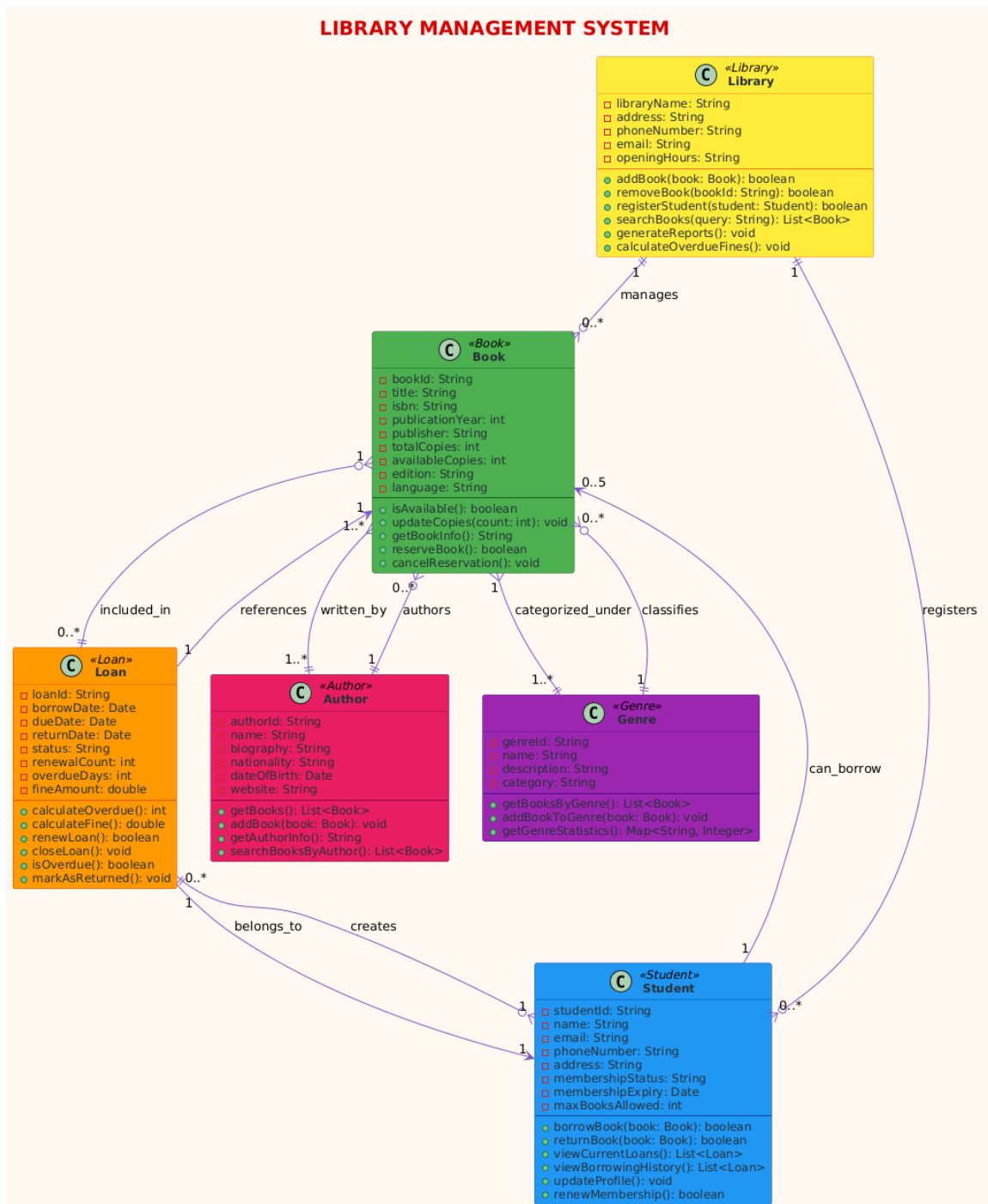


23P-0566 --- Waleed Afzal --- 5C -- CP --- SE

Diagram:



Code:

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// Base Class
class Person {
protected:
    string name;
    string email;
    string phoneNumber;

public:
    Person(string n, string e, string p) : name(n), email(e), phoneNumber(p) {}

    virtual void displayInfo() {
        cout << "Name: " << name << "\nEmail: " << email << "\nPhone: " <<
phoneNumber << endl;
    }

    string getName() { return name; }
};

// Derived Class - Student
class Student : public Person {
private:
    string studentId;
    string membershipStatus;

public:
```

```

Student(string id, string n, string e, string p, string status)
    : Person(n, e, p), studentId(id), membershipStatus(status) {}

void displayInfo() override {
    cout << "=== STUDENT INFO ===" << endl;
    Person::displayInfo();
    cout << "Student ID: " << studentId << "\nMembership: " << membershipStatus
<< endl;
}

void borrowBook() {
    cout << name << " is borrowing a book..." << endl;
}

void returnBook() {
    cout << name << " is returning a book..." << endl;
}

};

// Derived Class - Author
class Author : public Person {
private:
    string authorId;
    string nationality;

public:
    Author(string id, string n, string e, string p, string nation)
        : Person(n, e, p), authorId(id), nationality(nation) {}

    void displayInfo() override {
        cout << "=== AUTHOR INFO ===" << endl;
        Person::displayInfo();
    }
}

```

```
        cout << "Author ID: " << authorId << "\nNationality: " << nationality << endl;
    }
}
```

```
void writeBook() {
    cout << name << " is writing a new book..." << endl;
}

};
```

// Book Class

```
class Book {
private:
    string bookId;
    string title;
    string isbn;
    int publicationYear;

public:
    Book(string id, string t, string i, int year)
        : bookId(id), title(t), isbn(i), publicationYear(year) {}

    void displayBookInfo() {
        cout << "=== BOOK INFO ===" << endl;
        cout << "Title: " << title << "\nISBN: " << isbn
            << "\nYear: " << publicationYear << "\nID: " << bookId << endl;
    }

    string getTitle() { return title; }
};
```

// Genre Class

```
class Genre {
```

```

private:
    string genreId;
    string name;
    string description;

public:
    Genre(string id, string n, string desc)
        : genreId(id), name(n), description(desc) {}

    void displayGenre() {
        cout << "Genre: " << name << " - " << description << endl;
    }
};

// Loan Class
class Loan {
private:
    string loanId;
    string borrowDate;
    string dueDate;

public:
    Loan(string id, string borrow, string due)
        : loanId(id), borrowDate(borrow), dueDate(due) {}

    void displayLoanInfo() {
        cout << "=== LOAN INFO ===" << endl;
        cout << "Loan ID: " << loanId << "\nBorrowed: " << borrowDate
            << "\nDue: " << dueDate << endl;
    }

    void calculateFine() {

```

```

        cout << "Calculating fine for loan " << loanId << "..." << endl;
    }
};

// Library Class (Main Class)
class Library {
private:
    string libraryName;
    string address;

public:
    Library(string name, string addr) : libraryName(name), address(addr) {}

    void displayLibraryInfo() {
        cout << "=== LIBRARY INFO ===" << endl;
        cout << "Library: " << libraryName << "\nAddress: " << address << endl;
    }

    void addBook(Book book) {
        cout << "Adding book: " << book.getTitle() << " to library..." << endl;
    }

    void registerStudent(Student student) {
        cout << "Registering student: " << student.getName() << "..." << endl;
    }
};

// Main Function to demonstrate the system
int main() {
    cout << " LIBRARY MANAGEMENT SYSTEM 📖" << endl;
    cout << "=====\n" << endl;

```

```

// Create Library
Library centralLib("Central Library", "123 Main Street");
centralLib.displayLibraryInfo();
cout << endl;

// Create Students (inherited from Person)
Student student1("S001", "John Doe", "john@email.com", "123-4567", "Active");
Student student2("S002", "Jane Smith", "jane@email.com", "987-6543", "Active");

// Create Authors (inherited from Person)
Author author1("A001", "Robert Writer", "robert@author.com", "555-1234",
"American");
Author author2("A002", "Emily Poet", "emily@author.com", "555-5678",
"British");

// Create Books
Book book1("B001", "C++ Programming", "978-123456", 2023);
Book book2("B002", "Data Structures", "978-789012", 2024);

// Create Genre
Genre programming("G001", "Programming", "Computer programming books");

// Create Loan
Loan loan1("L001", "2024-01-15", "2024-02-15");

// Demonstrate inheritance
cout << " PERSON DEMONSTRATION (INHERITANCE):" << endl;
cout << "-----" << endl;

// Using base class pointer to demonstrate polymorphism
Person* people[4];

```

```

people[0] = &student1;
people[1] = &student2;
people[2] = &author1;
people[3] = &author2;

for (int i = 0; i < 4; i++) {
    people[i]->displayInfo();
    cout << endl;
}

// Demonstrate other functionalities
cout << " BOOK OPERATIONS:" << endl;
cout << "-----" << endl;
book1.displayBookInfo();
cout << endl;

cout << " GENRE INFORMATION:" << endl;
cout << "-----" << endl;
programming.displayGenre();
cout << endl;

cout << " LOAN OPERATIONS:" << endl;
cout << "-----" << endl;
loan1.displayLoanInfo();
loan1.calculateFine();
cout << endl;

cout << " LIBRARY OPERATIONS:" << endl;
cout << "-----" << endl;
centralLib.addBook(book1);
centralLib.registerStudent(student1);
cout << endl;

```

```
cout << STUDENT SPECIFIC OPERATIONS:" << endl;
cout << "-----" << endl;
student1.borrowBook();
student1.returnBook();
cout << endl;

cout << " AUTHOR SPECIFIC OPERATIONS:" << endl;
cout << "-----" << endl;
author1.writeBook();

return 0;
}
```